

Kode ini adalah sebuah fungsi pemrosesan gambar (Image Processing) yang ditulis dalam bahasa C# menggunakan library GDI+ (System.Drawing).

Tujuan Kode:

Fungsi ApplySelection ini bertujuan untuk mengisolasi warna tertentu dari sebuah gambar.

- Jika pixel pada gambar asli sesuai dengan warna yang dipilih (selectedColor), pixel tersebut dipertahankan.
- Jika tidak sesuai, pixel tersebut diubah menjadi putih.
- Hasilnya adalah gambar baru yang hanya menampilkan objek dengan warna yang dipilih, sementara sisanya menjadi latar belakang putih.

Metode ini menggunakan **Direct Memory Access (LockBits)** yang jauh lebih cepat daripada menggunakan metode GetPixel dan SetPixel biasa.

Berikut adalah penjelasan detail baris per baris:

1. Import dan Namespace

C#

```
using System.Drawing.Drawing2D;
using System.Drawing.Imaging;
using System.Runtime.InteropServices;

namespace MiniPhotoshop.Logic.ImageProcessing
{
    public static class SelectionColor
    {

```

- **System.Drawing...:** Mengimpor pustaka grafis untuk memanipulasi Bitmap dan Pixel.
- **System.Runtime.InteropServices:** Mengimpor pustaka untuk layanan Marshal, yang digunakan untuk menyalin data antara memori managed (array C#) dan unmanaged (pointer memori gambar).
- **namespace & class:** Mendefinisikan kelas statis SelectionColor agar metode di dalamnya bisa dipanggil langsung tanpa membuat objek (instansiasi).

2. Definisi Fungsi dan Persiapan

C#

```
public static Bitmap ApplySelection(Bitmap originalImage, Color selectedColor)
{
    Bitmap resultImage = new Bitmap(originalImage.Width, originalImage.Height,
    originalImage.PixelFormat);
```

- Mendefinisikan fungsi ApplySelection yang menerima gambar asli (originalImage) dan warna target (selectedColor).
- **resultImage**: Membuat kanvas gambar baru yang kosong dengan lebar, tinggi, dan format pixel yang sama persis dengan gambar asli. Ini akan menjadi wadah hasil edit.

C#

```
int targetArgb = selectedColor.ToArgb();
Rectangle rect = new Rectangle(0, 0, originalImage.Width, originalImage.Height);
```

- **targetArgb**: Mengonversi objek Color yang dipilih menjadi satu angka integer 32-bit (ARGB). Ini dilakukan agar perbandingan warna nanti lebih cepat (membandingkan integer vs integer, bukan objek vs objek).
- **rect**: Membuat kotak persegi panjang seukuran gambar penuh. Ini digunakan untuk memberi tahu area mana yang akan kita kunci (lock) di memori.

3. Mengunci Memori (LockBits)

C#

```
BitmapData originalData = originalImage.LockBits(rect, ImageLockMode.ReadOnly,
originalImage.PixelFormat);
BitmapData resultData = resultImage.LockBits(rect, ImageLockMode.WriteOnly,
resultImage.PixelFormat);
```

- **LockBits**: Ini adalah langkah krusial untuk performa. Kita "mengunci" data pixel gambar

di memori RAM agar Garbage Collector tidak memindahkannya saat kita sedang memproses.

- **originalData**: Dikunci dalam mode **ReadOnly** (hanya baca).
- **resultData**: Dikunci dalam mode **WriteOnly** (hanya tulis).

C#

```
IntPtr ptrAsli = originalData.Scan0;  
IntPtr ptrHasil = resultData.Scan0;
```

- **Scan0**: Mendapatkan alamat memori (pointer) dari pixel pertama (kiri atas) gambar. **ptrAsli** menunjuk ke data gambar sumber, **ptrHasil** menunjuk ke data gambar hasil.

4. Menyalin Data ke Array (Marshalling)

C#

```
int bytes = Math.Abs(originalData.Stride) * originalImage.Height;  
byte[] rgbAsli = new byte[bytes];  
byte[] rgbHasil = new byte[bytes];
```

```
Marshal.Copy(ptrAsli, rgbAsli, 0, bytes);
```

- **Stride**: Lebar satu baris gambar dalam satuan *byte* (bukan pixel). Kadang ada *padding* kosong di ujung baris agar memori sejajar. Total byte = Stride x Height.
- **rgbAsli & rgbHasil**: Kita membuat array `byte[]` di C# untuk menampung seluruh data pixel.
- **Marshal.Copy**: Menyalin seluruh data mentah dari pointer memori gambar (**ptrAsli**) ke dalam array C# (**rgbAsli**). Sekarang kita bisa mengedit array ini dengan sangat cepat.

5. Loop Pemrosesan Pixel

C#

```
int bytesPerPixel = Image.GetPixelFormatSize(originalImage.PixelFormat) / 8;
int stride = originalData.Stride;
```

- **bytesPerPixel**: Menentukan berapa byte yang dipakai 1 pixel.
 - Jika format 32bpp (ARGB), nilainya 4.
 - Jika format 24bpp (RGB), nilainya 3.
- **stride**: Disimpan dalam variabel lokal untuk akses lebih cepat di dalam loop.

C#

```
for (int y = 0; y < originalImage.Height; y++)
{
    int rowOffset = y * stride;
    for (int x = 0; x < originalImage.Width; x++)
    {
        int i = rowOffset + (x * bytesPerPixel);
```

- Melakukan loop ganda: y untuk baris (vertikal), x untuk kolom (horizontal).
- **rowOffset**: Menghitung indeks awal baris saat ini dalam array byte.
- **i**: Menghitung posisi indeks byte tepat untuk pixel pada koordinat (x, y).

6. Membaca Komponen Warna (Penting!)

C#

```
byte b = rgbAsli[i];
byte g = rgbAsli[i + 1];
byte r = rgbAsli[i + 2];
byte a = (bytesPerPixel == 4) ? rgbAsli[i + 3] : (byte)255;
```

- Mengambil nilai warna Merah, Hijau, Biru, dan Alpha.
- **PENTING**: Urutan byte dalam memori Windows Bitmap biasanya adalah **BGR** (Blue, Green, Red), bukan RGB.
 - **i**: Blue

- i+1: Green
- i+2: Red
- Jika gambar memiliki Alpha channel (transparansi/4 byte), kita ambil dari i+3. Jika tidak (3 byte), kita anggap Alpha-nya penuh (255/tidak transparan).

C#

```
int currentArgb = (a << 24) | (r << 16) | (g << 8) | b;
```

- **Bitwise Operation:** Menggabungkan 4 byte terpisah (A, R, G, B) kembali menjadi satu angka integer 32-bit (currentArgb).
 - a digeser 24 bit ke kiri.
 - r digeser 16 bit ke kiri.
 - g digeser 8 bit ke kiri.
 - b tetap di posisi paling kanan.
 - Operator | (OR) menggabungkan semuanya.

7. Logika Seleksi Warna

C#

```
if (currentArgb == targetArgb)
{
    rgbHasil[i] = b;
    rgbHasil[i + 1] = g;
    rgbHasil[i + 2] = r;
    if (bytesPerPixel == 4) rgbHasil[i + 3] = a;
}
```

- **Jika Cocok:** Jika warna pixel saat ini (currentArgb) SAMA persis dengan warna yang dicari (targetArgb), maka salin nilai B, G, R, A asli ke array hasil (rgbHasil). Pixel dipertahankan.

C#

```

else
{
    rgbHasil[i] = 255;
    rgbHasil[i + 1] = 255;
    rgbHasil[i + 2] = 255;
    if (bytesPerPixel == 4) rgbHasil[i + 3] = a;
}
}
}

```

- **Jika Tidak Cocok:** Isi array hasil dengan nilai **255**.
 - R=255, G=255, B=255 menghasilkan warna **Putih**.
 - Alpha (a) tetap disalin agar transparansi gambar asli (jika ada) tidak rusak, atau bisa diset 255 juga tergantung kebutuhan.

8. Finalisasi dan Pembersihan

C#

```

Marshal.Copy(rgbHasil, 0, ptrHasil, bytes);

originalImage.UnlockBits(originalData);
resultImage.UnlockBits(resultData);

return resultImage;
}

```

- **Marshal.Copy (Balik):** Menyalin data dari array rgbHasil (yang sudah kita modifikasi) kembali ke pointer memori gambar (ptrHasil). Sekarang gambar resultImage sudah berisi pixel-pixel baru.
- **UnlockBits:** Melepaskan kunci memori. Ini **wajib** dilakukan. Jika tidak, gambar akan tetap terkunci dan bisa menyebabkan kebocoran memori (memory leak) atau error saat gambar itu mau dipakai lagi.
- **return:** Mengembalikan gambar hasil editan.

Ringkasan Alur

1. Siapkan gambar kosong.

2. Salin seluruh data pixel gambar asli ke Array Byte (RAM).
3. Looping setiap pixel:
 - Susun ulang byte menjadi warna (ARGB).
 - Cek apakah warnanya sama dengan target?
 - Ya -> Salin warna asli.
 - Tidak -> Ubah jadi Putih.
4. Salin Array Byte hasil kembali ke Gambar Hasil.
5. Selesai.